Module 5 Day 1

PRE-WORK

SEMICONDUCTOR IC DESIGN FOUNDATION

AND READINESS PROGRAMME

2-Hour Training Module

Module 5 at a glance

ORIENTATION

Build practical familiarity with Verilog before participants begin using professional EDA tools. Participants will learn how digital hardware is described, simulated, tested and prepared for synthesis using a Hardware Description Language.

Learning outcomes

ORIENTATION

Basic structure of NMOS and PMOS transistors; gate, source and drain terminals; transistor switching operation; ON and OFF states; basic CMOS operation; how NMOS and PMOS transistors form logic gates; voltage, current and resistance fundamentals; threshold voltage; relationship between transistor behaviour, power consumption and circuit speed.

Introduction

Verilog HDL is one of the two most common Hardware Description Languages (HDL) used by integrated circuit (IC) designers. The other one is VHDL.

Designs described in HDL are technology-independent, easy to design and debug, and are usually more readable than schematics, particularly for large circuits.

Verilog generally requires more debugging effort than VHDL because it is a weakly-typed language.

Introduction

(i) Algorithmic level (much like c code with if, case and loop statements).
(ii) Register transfer level (RTL uses registers connected by Boolean equations).
(iii) Gate level (interconnected AND, NOR etc.).
(iv) Switch level (the switches are MOS transistors inside gates). The language also defines constructs that can be used to control the input and output of simulation.

Structure of HDL

Structural, which is a verbal wiring diagram without storage.

```
assign a=b & c | d; /* “|” is a OR */
assign d = e & (~c); Here the order of the statements does not matter.
```

Procedural which is used for circuits with storage, or as a convenient way to write conditional logic.

```
always @(posedge clk) // Execute the next statement on every rising clock edge.
count <= count+1;
```

Basic Gates (GATE stands for one of the keywords and, nand, or, nor, xor, xnor)

Syntax

```
GATE (drive_strength) # (delays)
instance_name1(output, input_1,
                        input_2,..., input_N),
instance_name2(outp,in1, in2,..., inN);
```

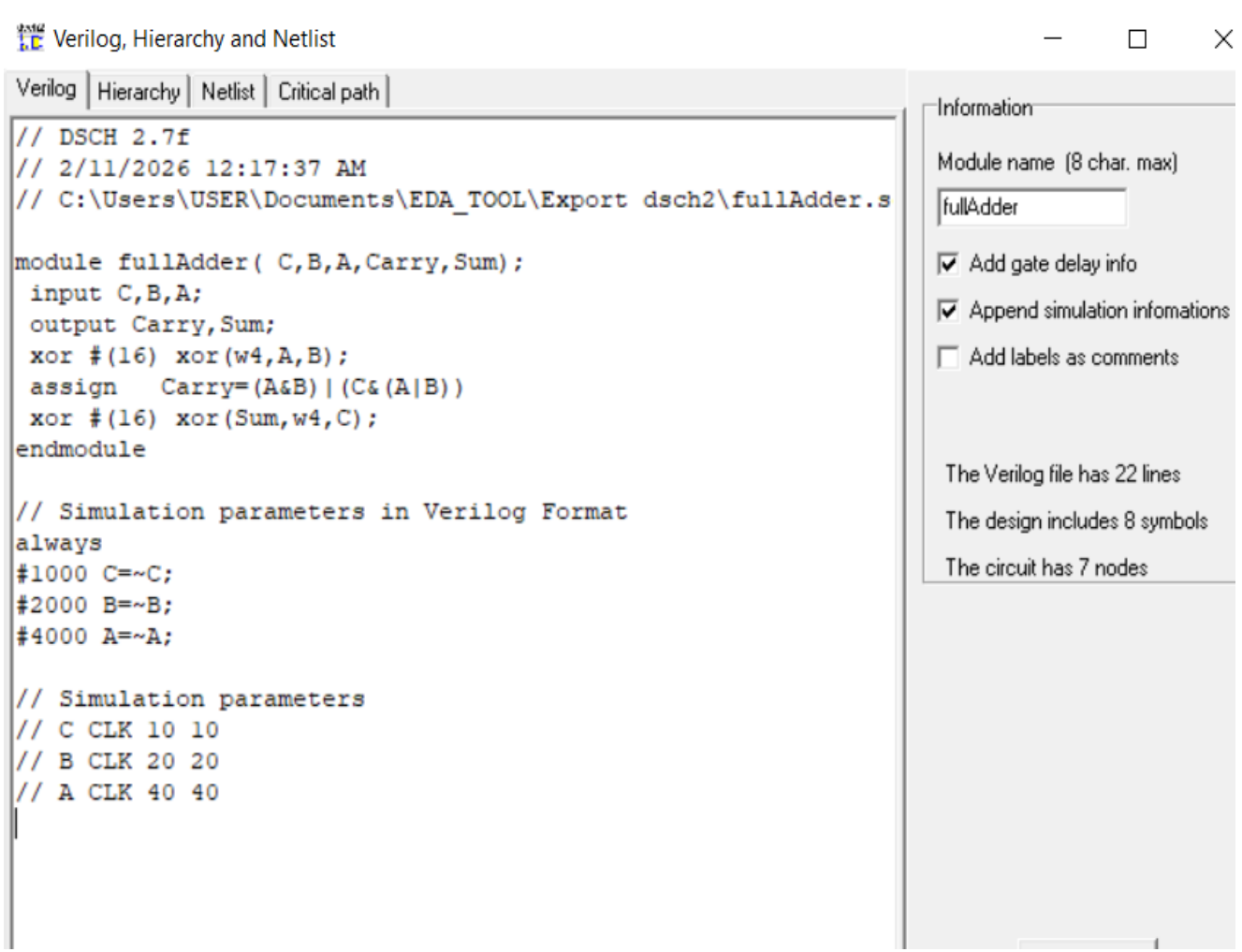
Delays is

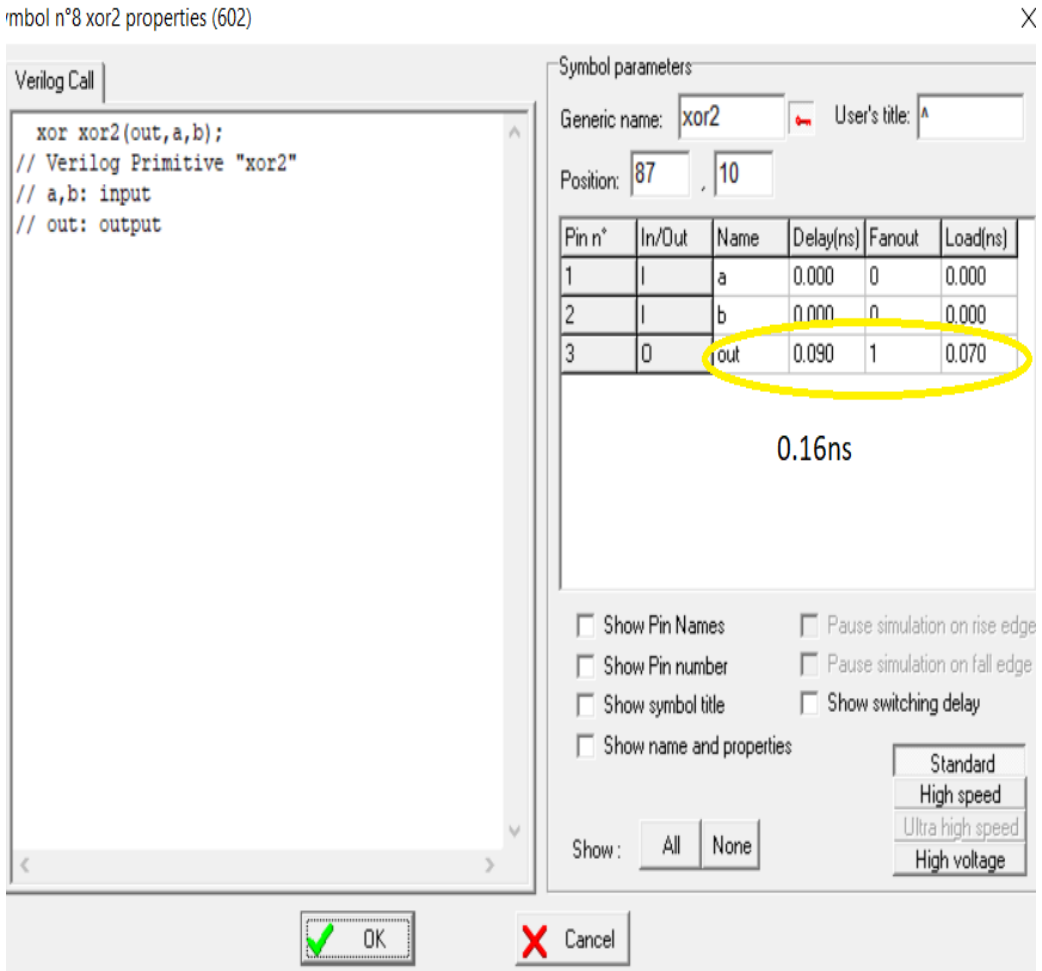
```
#(rise, fall)   or
# rise_and_fall     or
#(rise_and_fall)
```

Example 3 .1

```
and c1 (o, a, b, c, d);      // 4-input AND called c1 and
    c2 (p, f g);             // a 2-input AND called c2.
or #(4, 3) ig (o, a, b);     /* or gate called ig (instance name);
                                rise time = 4, fall time = 3  */
xor #(5) xor1 (a, b, c);     // a = b XOR c after 5 time units
xor (pull1, strong0) #5 (a,b,c);  /* Identical gate with pull-up
                                strength pull1 and pull-down strength strong0. */
```

Full adder and delay





Group Exercise (Day 1)

Task

1. - Please choose at least more than one gates (eg full adder). Study the verilog code.
2. Show the signal path of the circuit (Show at layout)
3. Explain the delay based on the timing diagram. Looking at the delay in verilog code.
4. Determine the current or power consume by the circuit
5. Repeat (3) and (4) for 180nm (cmos018) and 50 nm (cmos50n) processes.
6. Calculate the power based on the total capacitance and clock frequencies.

Knowledge check

WRAP-UP

1. Basic Gate Verilog code.
2. Assign for complex gate.
3. Delay can be added for functional simulation.

Module 5: submit report
https://drive.google.com/drive/folders/1pJiXCG217wpYqlUotU7CZ2s9T534I9iQ?usp=drive_link